Online Longitudinal Trajectory Planning for Connected and Autonomous Vehicles in Mixed Traffic Flow with Deep Reinforcement Learning Approach

Yanqiu Cheng[1,2], Xianbiao Hu[2*], Kuanmin Chen[1], Xinlian Yu[3]

1 Department of Traffic Engineering, College of Transportation Engineering, Chang'an University, Xi'an, Shaanxi, China

2 Department of Civil, Architectural and Environmental Engineering, Missouri University of Science and Technology, Rolla, MO

3 School of Transportation Engineering, Southeast University. Nanjing, China.

* Corresponding Author: xbhu@mst.edu

ABSTRACT

This manuscript presents an Adam optimization-based Deep Reinforcement Learning model for Mixed Traffic Flow control (ADRL-MTF), so as to guide Connected and Autonomous vehicle's (CAV) longitudinal trajectory on a typical urban roadway with signal-controlled intersections. The goal is to minimize fuel consumptions of the traffic platoon. Two improvements are made when compared with prior literatures. First, the common assumptions to simplify the problem solving, such as dividing a vehicle trajectory into several segments with constant acceleration/deceleration, are avoided, to improve the modeling realism. Second, built on the efficient Adam optimization and Deep Q-Learning, the proposed ADRL-MTF model avoids the enumeration of states and actions, and is computational efficient and suitable for real time applications. The mixed traffic flow dynamic is firstly formulated as a finite Markov decision process (MDP) model. Due to the discretization of time, space and speed, this MDP model becomes high-dimensional in state, and is very challenging to solve. We then propose a temporal difference-based deep reinforcement learning approach, with ε-greedy for exploration-exploitation balance. Two neural networks are developed to replace the traditional Q function and generate the targets in the Q-learning update. These two neural networks are trained by the Adam optimization algorithm, which extends stochastic gradient descent and considers the second moments of the gradients, and is thus highly computational efficient and has lower memory requirements. The proposed model is shown to reduce fuel consumption by 7.8%, which outperforms a prior benchmark model based on Monte Carlo Tree Search (MCTS). The model's runtime efficiency and stability are tested, and the sensitivity analysis is also performed.

Keywords: Connected and Automated Vehicles; Longitudinal Trajectory Planning; Deep Q-Learning; Reinforcement Learning;

1. INTRODUCTION

Connected and Automated Vehicles (CAVs) have been shown to increase traffic flow efficiency and enhance driving safety. When compared with Human Driven Vehicles (HDVs), one fundamental difference is CAV's movements are controlled, and thus can be scheduled, by the computer algorithms with pre-defined objectives such as minimizing fuel consumptions or maximizing traffic throughputs. Trajectory planning is the main task to achieve these goals (Eskandarian et al. 2019). It uses the on-board data (e.g., in-vehicle sensors, radar, camera, lidar, etc.) and the vehicle communication data (e.g. dedicated short-range communications, or Cellular, etc.) from the perception layer as the inputs for decision makings. Then, the computed CAV trajectory information is transferred to the control layer to move CAV forward (Guanetti et al. 2018). Despite of numerous existing researches, CAV trajectory scheduling remains a sophisticated problem to solve, as it is essentially an infinite-dimensional optimization problem considering the continuous and stochastic nature of human driving behaviors in the mixed traffic flow where CAV and HDV co-exist.

The computational complexity and algorithm runtime of CAV trajectory planning problem are directly related to the environment that CAV operates in. One way to reduce the complexity of the model is to only consider the pure CAV traffic. In fact, many researches on CAV trajectory planning are under this assumption. For example, Lee and Park developed a CVIC algorithm for manipulating individual automated vehicle to cross an intersection without colliding with other vehicles in an environment with 100% CAV market penetration rate (MPR) (Lee and Park 2012). Wang et al. (2014) proposed a rolling horizon control framework to control all vehicles' trajectory, which were equipped with driver assistance systems by optimizing a cost function reflecting different control objectives. Under the same assumption, Ahn et al. (2013) developed an eco-drive system that combined an eco-cruise control algorithm and state-of-the-art car-following models. Zhou et al. (2020) proposed a reinforcement-learning-based approach to train a CAVs platoon to passthrough the intersection at a steady speed. Similar research with a context of pure CAV environment can also be found in (Walraven et al. 2016, Cao et al. 2017, Mirheli et al. 2018, Mirheli et al. 2019, Liang et al. 2020, Liang et al. 2020). However, it is envisioned that mixed traffic flow, with both HDV and CAV, is inevitable as it is difficult to completely replace all HDVs in a short period of time.

Researches on CAV trajectory planning in the mixed traffic flow context can also be found in the literature. For example, Jiang at el. (2017) proposed an eco-driving system to improving fuel efficiency of mixed traffic and tested this system in different CAV's MPR. Yang et al. (2016) took into account three types of vehicles in intersection control and formulated it as a bi-level optimization problem. Ghiasi et al. (2019) proposed a CAV speed harmonization algorithm to predict each vehicle trajectory in a mixed traffic environment. One common modeling approach in mixed-flow control is to simplify the problem by dividing a vehicle trajectory into several segments, each with a fixed cruising speed or with a constant acceleration/deceleration value. For example, Wu et al. (2015) divided the whole vehicle control process into a sequence of control

stages, and each control stage was formulated as an individual optimal control problem involving spatial and temporal constraints induced by the presence of vehicle queues. He et al. (2015) proposed a multi-stage approximation control model to solve the optimal trajectory problem. First, the vehicle cruised at a speed calculated by their algorithm, and then accelerated/decelerated to a final speed when passing through the intersection. In Yao et al. (2018), the roadway was separated into three segments by two Individual Variable Speed Limits (IVSL). At those IVSL locations, vehicle speed was adjusted to a final constant value, so that their trajectories became smooth. In Liang et al. (2019), vehicles were assumed to accelerate to different optimal cruising speeds at some locations that divided the roadway. Similar method can be found in (Ma et al. 2017, Zhou et al. 2017), in which each vehicle trajectory was broken into a few sections to decompose the originally hard trajectory design problem to a simpler one. These models, although analytically solvable and reduced the computational burden, essentially sacrificed the modeling realism for runtime efficiency.

To capture the vehicle dynamics within the mixed traffic flow, we firstly formulate the CAV trajectory planning problem as a finite Markov decision process (MDP). We focus on a typical 1-lane roadway with signal-controlled intersections, and define the objective of CAV longitudinal trajectory planning to be minimizing fuel consumption of the total traffic flow. The common assumptions (i.e., dividing a vehicle trajectory into several segments, or assuming a pure CAV environment) to reduce simulation realism are avoided. We then decompose the mixed traffic with multiple CAVs and HDVs into several basic interactive units (BIUs), each with a leading CAV and potentially multiple following HDVs. The State, Action, Reward, and State transition probabilities of this MDP model are defined, with the HDV's car-following behavior on the roadway segment and vehicle dynamics when approaching intersections as the key components of the MDP environment model. Due to the discretization of time, space and speed, this MDP model becomes high-dimensional, and is thus very challenging to solve.

Solving this MDP model requires efficient ways to tackle this "curse of dimensionality" issue, and thus, an Adam optimization-based Deep Reinforcement Learning model for Mixed Traffic Flow control (ADRL-MTF) is developed for this purpose. Two neural networks are developed, including a Q-network that replaces the traditional Q function, and a target network that generates the targets in the Q-learning update. The exploration-exploitation dilemma of the reinforcement learning model is balanced by the ε-greedy algorithm. The Q network and target network are trained by the Adam optimization algorithm, which extends stochastic gradient descent and considers the second moments of the gradients, and is thus highly computational efficient and has lower memory requirements. The main input data required by the training model is the length of the roadway segment, speed limit, and the signal control information. In other words, as long as the basic information of a roadway segment is known, the proposed ADRL-MTF model can perform reinforcement learning by itself and identify the best actions for CAV along the way. We compare its performance with a prior benchmark model based on Monte Carlo Tree Search (MCTS) (Cheng et al. 2020), and also test the algorithm's stability with regards to different

headway distribution, as well as a variety of market penetration rates. Sensitivity analysis is also performed to test the algorithm's effectiveness with different roadway geometries.

This paper is organized as follows. Section 2 summarizes the notations used in this manuscript. Section 3 presents the MDP problem formulation, including the definition of State, Action, Reward, and the environment model. Section 4 presents the developed ADRL-MTF model to solve this MDP problem. The developed model is implemented and analyzed in Section 5, and its stability and sensitivity analysis are discussed. Section 6 concludes this paper.

2. NOTATIONS

As a convenient reference, the mathematical notations used in this section are presented below.

Parameters of MDP formulation

t, T: Discrete time step, and the time horizon

s, S: State and set of all States

$a, \mathcal{A}(\mathrm{s})$: Action of CAV, and set of all Actions at State *s*

$r, \mathcal{R}$: immediate Reward, and set of all possible Rewards

Parameters of environment model

y_i^t: the distance of vehicles i from roadway entrance, at time t

Y_t: an array that stores vehicles' distance from roadway entrance, at time t

v_i^t: the speed of vehicle i, at time *t*

V_t: an array that stores vehicles' speed, at time *t*

d_{hi}^t: the distance headway of vehicle *i*, at time *t*

D_t: an array that stores vehicles' distance headway, at time *t*.

t_g, t_y, t_r, t_c: durations of green, yellow, and red signals, and cycle length

c: an array that indicates signal color, $c \in [(1,0,0), (0,1,0), (0,0,1)]$, representing green, yellow and red, respectively

v_s: speed limit

l_{seg}: length of the roadway segment

l_a: average vehicle length

Parameters of DQN algorithm

θ, θ^-: parameters of a neural network

N: the number of training episodes

α: the learning rate

γ: the decay rate

3. MODEL FORMULATION

3.1 Problem setting and decomposition

We focus on the following general scenario: a mixed traffic flow platoon is composed of multiple HDVs and CAVs, and is traveling on a single lane arterial segment. An intersection with pre-fixed signal control is located at the end of this roadway. The mixed traffic may include one single CAV with multiple HDVs, or multiple CAVs with multiple HDVs as shown. Obviously, if there is only one CAV, the interaction between CAV and HDV becomes simple. The HDVs behaviors are only affected by the leading CAV, and could be described by car-following models. However, when multiple CAVs exist in this traffic flow, the car following behavior becomes complicated and difficult to describe analytically. In this case, we first decompose the mixed traffic flow into several "basic interactive unit (BIU)" as illustrated in Figure 1 below. Each BIU has a leading CAV and multiple following HDVs. For the example setup in Figure 1 below, based on the location of CAVs, a total of three basic interactive units (represented by dashed boxes) are obtained.

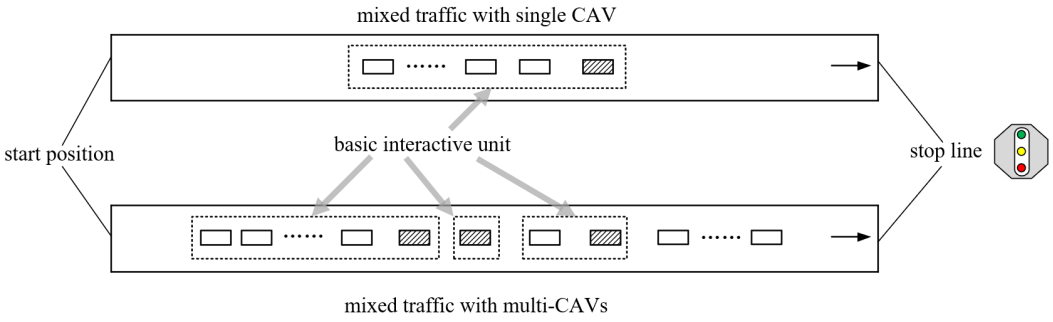


Figure 1. Decomposition of mixed traffic flow into basic interactive units

After the decomposition, each group has only one CAV that is leading the platoon, and the rest of the vehicles in the platoon are HDVs. The number of HDVs might be one or multiple, or there might be no CAV at all. As such, the mixed traffic flow problem can be converted into a trajectory optimization problem for each basic interaction unit, which significantly reduced the computational complexity. There are two reasons for such decomposition. First, if an HDV is driving in front of a CAV, due to the human nature, it will drive according to speed limit or prevailing cruising speed, and as a result, its behavior is not impacted by the CAV behind it. Second, CAV is subject to the speed limit or current traffic conditions, as such it cannot drive faster than a typical HDV. On the other hand, when it slows down to a speed that is lower than HDV, it becomes a moving bottleneck and all HDVs behind it are forced to slow down and follow this CAV. To summarize, for mixed traffic flow control problem, we will always have a CAV that is leading the platoon, and potentially multiple HDVs behind the CAV, in each basic interactive unit. Such decomposition is also frequently used in the previous literatures.

3.2 Markov decision process of mixed traffic flow

3.2.1 MDP framework

In this section, we show how to model the CAV decision makings in each basic interactive unit as a Markov decision process (MDP). Due to the definition of "connected" and "autonomous", at each time step t, in the mixed traffic flow, CAV collects data from the surrounding environment to enable the situational awareness, and subsequently, chooses an action to achieve its pre-defined objective. The behavior of CAV is only based on the current state of this mixed traffic flow. In other words, CAV determines its next move based on the information that it has at the current moment. If we discretize the time dimension, the performance of this mixed traffic platoon can be calculated by the summation of the benefit value at each time interval t. CAV trajectory planning essentially becomes a sequential decision-making process, as shown in Figure 2 below.

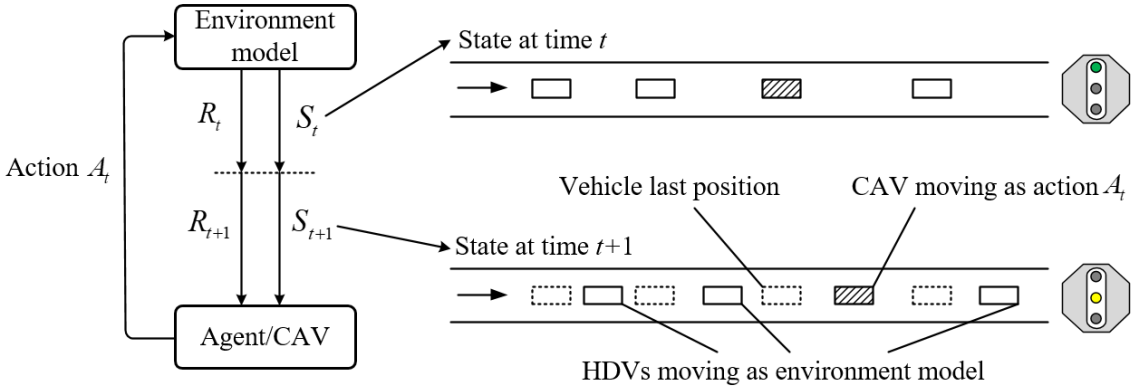


Figure 2. MDP framework for mixed traffic flow control

In this MDP framework, CAV is the agent that needs to determine its best action, while the mixed traffic flow is the environment that CAV operates in. The environment model generates two main outputs at each time step t: 1) state characteristic value S_t, in the form of a vector to represent the relevant information of CAV's external driving context, and 2) a reward characteristic value R_t, to describe how has the system performed so far. At any time t, based on S_t and R_t, the agent (i.e., CAV) chooses an action a from action space A_t (i.e., $a \in A_t$) to move itself. The HDVs in the surrounding environment will also make a move (e.g., maintaining current speed, or accelerating/decelerating) simultaneously. For example, in Figure 2, the upper right picture shows the vehicle's position at time t (each box represent a vehicle, with the shaded box being CAV), whereas in the lower right picture, their locations at time $t+1$ move towards the intersection (dashed box represent vehicle location at time t, and solid line box represent vehicle location at time $t+1$). The other elements in the environment, such as the signal phase and remaining signal duration will also be updated. With these updates, the current state S_t transitions to a new state S_{t+1}, and generates a new reward R_{t+1}. Together they complete a state transition of the mixed platoon movement, and such process repeats until the entire vehicle platoon passes the intersection.

3.2.2 State definition

At a given time t, the state of MDP problem S_t is defined by a list of specific input features to describe the current system status, and is required for any reinforcement learning algorithm. For a mixed traffic flow, many variables can be used to describe the state, for example, vehicle's distance from roadway entrance, vehicle velocity, accelerations, spacing/time headways between vehicles, elapsed time, signal light color and their remaining duration and so on. Obviously, when more features are selected, more details of the state will be captured. However, excessive number of state elements may directly lead to an exponential growth of the state space and lead to the "curse of dimensionality". As a result, a huge state space will come with a higher memory requirement and computational burden. Therefore, the features have to be chosen carefully.

Eq. (1) shows the four features selected to define the MDP state of the studied problem. The definitions of each variable are provided in Table 1, together with their units. The first feature c is the color of traffic light, with (1,0,0), (0,1,0) and (1,0,1) representing green, yellow, and red, respectively. $\frac{t_{remaining}}{t_c}$ is the percentage of remaining phase duration of the entire signal cycle. $1-\frac{y_{CAV}^t}{l_{seg}+l_a}$ is the percentage of remaining distance for CAV to travel until reaching intersection, and $\frac{v_{CAV}^t}{v_s}$ is the ratio of CAV speed to the speed limit. When combined together, these four features provide a holistic picture of CAV's driving context, including its location, speed, distance to travel, as well as signal information. Min-Max normalization is performed in Eq. (1), so as to ensure that the range of each feature is strictly within (0,1).

$$input\ features\ tuple = \left(c, \frac{t_{remaining}}{t_c}, 1-\frac{y_{CAV}^t}{l_{seg}+l_a}, \frac{v_{CAV}^t}{v_s}\right) \quad \text{Eq. (1)}$$

Table 1. Input features value

Features	Meaning	Values/ fundamental unit
c	The color of signal at time *t*	green: (1,0,0) yellow: (0,1,0) red: (0,0,1)
$t_{remaining}$	Remaining time of the color at time *t*	sec
$l_{seg}+l_a-y_{CAV}^t$	The distance from the CAV position before passing the stop line, add description of the equation	m
v_{CAV}^t	CAV velocity at time *t*	m/s

It is assumed that CAV is aware of all information with the help of the connected vehicle technology. Here, in terms of "connectivity", we mostly consider the connectivity between CAV and infrastructure (V2I), including the communication with traffic signal and the roadside units. In other words, the CAV is aware of the traffic signal timing plan in real time, including which

light is on and how many seconds are left until the phase changes. The CAV is also aware of its distance to the downstream intersection in real-time. Additionally, the CAV is able to detect the speed and distance headway of the HDVs immediately behind it, so that it can be used as input to decide its next move.

3.2.3 Action and State transitions

The action of a CAV (i.e., the agent) A_t is defined as its acceleration value, so when acceleration is zero, the CAV maintains its existing speed. When acceleration is a positive value, the CAV is speeding up, whereas when a negative value is observed, the CAV will slow down. We discretize the acceleration value, so the number of possible actions at time t is limited, with $A_t \in \mathcal{A}(s)$. $\mathcal{A}(s)$ is the set of all actions at State *s*.

In terms of the State transitions $p(s', r|s, a)$, due to the stochastic nature of the human driving behavior, one State may lead to multiple possible following States. For example, in Figure 2, when CAV makes one move, the following HDVs may exhibit different driving behavior and thus, the next State may not be unique for this stochastic MDP. Therefore, the state transition probability can be expressed as:

$$p(s', r|s, a) = P\{S_{t+1} = s', R_{t+1} = r|S_t = s, A_t = a\} \quad \text{Eq. (2)}$$

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r|s, a) = 1 \quad \text{Eq. (3)}$$

for all $s, s' \in \mathcal{S}, r \in \mathcal{R}, a \in \mathcal{A}(s)$. The state-transition probabilities $p(s', r|s, a)$ characterize the dynamics of a finite MDP of each episode.

3.2.4 Reward definition

Before defining the reward function R_i, we divide the states of this MDP into three types: normal terminal state, abnormal terminal state, and nonterminal state. Normal terminal state means the execution of this step will allow all vehicles to successfully pass intersection, and thus terminates the learning process. Abnormal terminal state means the learning process will be terminated due to vehicle crash or other kinds of traffic rule violation, whereas nonterminal state means this is merely an intermediate state of the learning process.

We start from the nonterminal state, and use Eq. (4) to calculate its reward R_i. In the equation below, B_i is the platoon benefit (i.e., system performance). Depending on the system objective, the optimization target of CAV movement might be the minimization of travel time (TT), or fuel consumption (FC). In this manuscript, our objective is to minimize fuel consumption, so B_i takes the form of FC_i, which can be calculated from VT-micro vehicle fuel consumption and emissions model (Ahn et al. 2013), in the form of $FC_i = \sum_{j=1}^{M} exp\left(\sum_{u=0}^{3} \sum_{w=0}^{3} K_{uw} v(t)_j^u a(t)_j^w\right)$, where $v(t)_j$ denotes the velocity of vehicle j $(j = 1, 2, \ldots, M)$ at time t, $a(t)_j$ denotes the acceleration of vehicle j (j=1, 2, …, m) at time t, and K_{uw} is a constant coefficient (Ma et al. 2017).

K_1 is a parameter to ensure the reward is always a positive value, as such we add a

constraint that K_1 should be not less than the value of B_i at state i. K_2 is another parameter to ensure the reward is always less than 1, as such, we add another constraint that K_2 should be not less than the summation of all reward values within an episode. For example, if the total fuel consumption of an episode is 860 ml, the value of K_2 could be set to 1,000, so as to keep R_i always less than 1. To summarize, with the introduction of K_1 and K_2, the reward function R_i is guaranteed to be strictly in the range of [0,1]. When the fuel consumption increases, the reward will be reduced, and the action associated with this reward will be less likely to be chosen as the CAV policy.

$$R_i = (K_1 - B_i)/K_2 \ , B_i = FC_i$$
$$K_1 > \max_i\{B_i\}, \forall i \qquad \text{Eq. (4)}$$
$$K_2 > \sum_{i=0}^{T} R_i$$

For the normal terminal state, since the last vehicle of the platoon has passed the stop line, we let $R_i = 1$ (i.e., the highest possible reward value) to encourage the successful completion of the learning process. On the other hand, in the event of abnormal terminal state, we let $R_i = -1$ to discourage such scenarios. The complete list of scenarios and associated reward values are listed in Table 2.

Table 2. Reward function

State type	Reward	Benefit
Nonterminal state	$(K_1 - B_i)/K_2$	$-FC_i$
Normal terminal state	1	$-FC_i$
Abnormal terminal state	-1	0

3.2.5 Optimal policy

As mentioned above, the objective function of this MDP is the minimization of fuel consumption FC, and the reward is defined in Table 2. As shown in Figure 2, at any given time t, CAV will assess the current state S_t and reward R_t, and chooses an action a from action space A_t (i.e., $a \in A_t$) to move itself, with the goal of maximizing the reward. The state-value function $v_\pi(s)$, i.e., the value of state *s* under a policy π is given in Eq. (5).

$$v_\pi(s) = \mathbb{E}_\pi\left[\sum_{k=0}^{T} \gamma^k R_{t+k+1} | S_t = s\right]$$
$$= \sum_a \pi(a|s) \sum_{s',r} p\left(s', r|s, a\right)\left[r + \gamma v_\pi(s')\right] \qquad \text{Eq. (5)}$$

And the action-value function $q_\pi(s, a)$, i.e., the value of taking action *a* in state *s* under a

policy π, can be calculated by Eq. (6).

$$q_\pi(s,a) = \mathbb{E}_\pi\left[\sum_{k=0}^{T}\gamma^k R_{t+k+1}|S_t = s, A_t = a\right] = \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi(s')\right] \quad \text{Eq. (6)}$$

For all $s, s' \in \mathcal{S}, r \in \mathcal{R}, a \in \mathcal{A}(s)$. $\gamma \in (0,1)$ is a decay parameter to ensure that both $v_\pi(s)$ and $q_\pi(s,a)$ can converge. To achieve the optimal condition, CAV's optimal policy can be derived by $\pi^*(a|s) = argmax_a q_\pi(s,a)$. In other words, CAV should select the action that has the highest value at each state. This value satisfies the Bellman optimality equations, and $v^*(s)$ and $q^*(s,a)$ can be derived by Eq. (7) and Eq. (8).

$$v^*(s) = max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1})|S_t = s, A_t = a] = max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_*(s')] \quad \text{Eq. (7)}$$

$$q^*(s,a) = max_a \mathbb{E}[R_{t+1} + \gamma max_{a'} q_*(S_{t+1}, a')|S_t = s, A_t = a] = \sum_{s',r} p(s',r|s,a)[r + \gamma max_{a'} q_*(s',a')] \quad \text{Eq. (8)}$$

3.3 MDP Environment model

This section presents the environment model that updates the MDP environment. The structure of the environment model and its input and output elements are illustrated in Figure 3. It should be noted that CAV action a is not a mandatory input, as HDVs can still drive normally without CAV's presence. The environment model consists of three modules: HDVs behavior, reward calculation and state termination conditions. The reward and state termination condition have both been defined earlier, so we will only focus on the HDV behavior below.

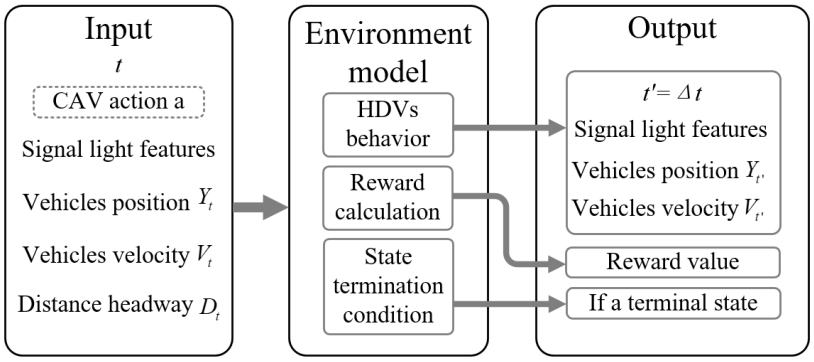


Figure 3. Environment model structure

In this MDP, the vehicles' positions (i.e., distance away from the roadway entrance) and

their velocities need to be updated at each time step. Since the CAV's movement is determined by the MDP, in the environment model we only need to focus on the movements of HDV vehicles. For HDVs in the traffic flow, there are two distinct scenarios: 1) when HDVs are relatively far away from the intersection, their behaviors are mostly car-following (CF) and can be described by classic CF model; 2) when HDVs are getting close to the intersection, the vehicle behaviors are subject to the signal lights. In other words, vehicles will drive through the intersection when the light is green, or if they cannot come to a safe stop when the yellow light is on. Otherwise it will slowdown and stop before the stop line. The HDVs behavior of these two scenarios are illustrated in Figure 4-(a) and (b), and both of them follow the vehicle constraints, including collision avoidance, speed limit, as well as vehicle kinematics.

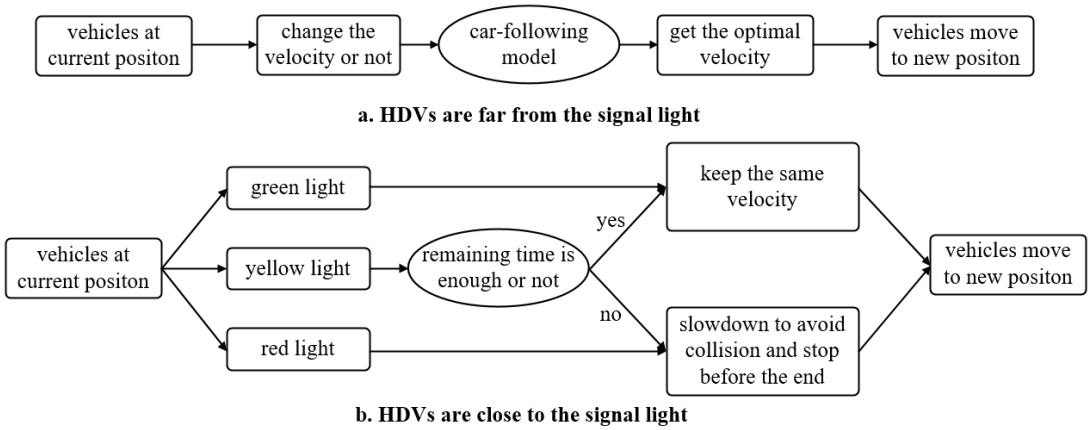


Figure 4. HDVs behavior in roadway with signal control intersection

To describe the velocity decision-making of the HDVs for the first scenario, the general GM model considering stochastic HDVs behavior is employed.

$$acc_i(t) = \alpha * v_i(t)^{\beta} \frac{\Delta v_i(t - t_{reaction})}{\Delta x_i(t - t_{reaction})^{\gamma}} + \epsilon_i(t) \quad \text{Eq. (9)}$$

Where $acc_i(t)$ is the acceleration value of the vehicle i at time t, $v_i(t)$ is the vehicle's speed. $t_{reaction}$ denotes the human perception reaction time, $\Delta v_i(t - t_{reaction})$ is the speed difference between the target vehicle and its leading vehicle at time $(t - t_{reaction})$, and $\Delta x_i(t - t_{reaction})$ is the space headway. α, β, γ are the parameters to calibrate. $\epsilon_i(t)$ is a random term associated with vehicle i at time t. Several studies (e.g. Gazis et al. 1961, Harmut 1967, Ahmed 1999) calibrated these parameters with collected data in real world.

4. SOLUTION ALGORITHM

Q-learning algorithm is usually employed to solve MDP problem. In our context, $Q(S_t, A_t)$ represents the value of CAV action A of state S at time t, and will be updated via Eq. (10). α is the learning rate, and $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$ is the time-difference target in Q-learning

algorithm.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \quad \text{Eq. (10)}$$

Solving this MDP problem with an explicit application of Q-learning algorithm turns out to be impossible. This is because Q-learning requires the enumeration and calculation of each $Q(S_t, A_t)$, in order to identify the optimal policy. However, learning from Eq. (1) and Table 1, the States S_t has four variables, three of which are continuous. In addition, the action A_t which represents CAV's acceleration value, is also a continuous variable. Effectively, this means we have unlimited numbers of combination of (S_t, A_t) and, even after discretization, this MDP model remains high-dimensional, and is very challenging to solve. As such, Adam optimization-based Deep Reinforcement Learning model for Mixed Traffic Flow control, or ADRL-MTF model, which utilizes a temporal difference-based deep Q-network (DQN) approach with ε-greedy for exploration-exploitation balance, is proposed below to solve this problem.

4.1 DQN formulation approach

We start off by building two back propagate neural networks (BPNN) with a same structure. The first is named Q-network to replace the Q function in Eq. (10), via a non-linear function approximator to solve this large-scale MDP problem. The second is named target network and is used to generate the targets in the Q-learning update. With these two neural networks, the Q function $Q(S_t, A_t)$ and optimal target $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$ are substituted with approximate Q function $Q(S_t, A_t; \theta)$ and target value $y = R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta^-)$, where θ and θ^- are parameters to be trained in the Q-network and target network, respectively. The loss function of these two BPNNs are defined by Eq. (11), and its gradient descent can be calculated by Eq. (12), in order to optimize parameters θ and θ^-.

$$L(\theta) = \mathbb{E}_{s_t, a_t, r_t} \left[\left(y - Q(S_t, A_t;\ \theta) \right)^2 \right] \quad \text{Eq. (11)}$$

$$\nabla_\theta L(\theta) = \mathbb{E}_{s_t, a_t, r_t, s_{t+1}} \left[\left(y - Q(S_t, A_t;\ \theta) \right) \nabla_\theta Q(S_t, A_t;\ \theta) \right] \quad \text{Eq. (12)}$$

Essentially, the introduction of Q-network and target network avoids the explicit estimation of reward R_t and state transition probability $p(s', r|s, a)$. Instead, it relies on the training data that are sampled from the environment model. Compared with the tabular solution methods from standard Q-learning method, this approach significantly reduces the required computer memory and improves runtime efficiency.

4.2 ε-greedy for exploration-exploitation balance

In the training process, CAV usually chooses the action with the highest Q-value to interact with the mixed traffic flow environment, i.e., the exploitation process. However, if CAV always behaves

in this way, many states may not be visited even though they may be promising, and CAV may be trapped at local optimum. On the other hand, if CAV focuses too much on the exploration process, i.e., with too many attempts on the entire sample space, many unreasonable states may be visited which substantially reduces the training efficiency and extends the training time, but the algorithm remains difficult to converge. As such, we design the following ε-greedy to balance the exploration and exploitation.

$$a_t = \begin{cases} \underset{a}{argmax}\, Q(S_{t+1}, a; \theta), & random\ number \leq \varepsilon \\ random\ action, & else \end{cases} \quad \text{Eq. (13)}$$

$$\varepsilon = max\left(\varepsilon_{lower}, \varepsilon_{upper} - \mu^{n/N}\right)$$

Where ε is a variable between 0 and 1, and is used to determine the degree of exploration. ε_{lower} denotes the lower bounds of ε to ensure that CAV will always have the ability to explore environment, whereas ε_{upper} denotes the upper bounds of ε for CAV to explore the sample space in the initial stage of training. μ is a hyper-parameter to control the descending speed of ε. n is the index of current iteration, and N is the total number of iterations. Eq. (12) ensures that at the beginning, ε is relatively high and the algorithm places higher weights on exploration. However, as the number of iterations n increases, the value of ε decreases and thus the algorithm turns to focus more on the exploitation.

4.3 Adam optimization algorithm for NN training

The Adam optimization algorithm is an extension to stochastic gradient descent that has recently seen broader adoption for deep learning applications. Instead of using the classical stochastic gradient descent procedure, it updates network weights iteratively based on training data. It combines the advantages of an adaptive gradient algorithm and root mean square propagation with momentum (Kingma and Ba 2015). Instead of updating learning rates based on the average first moment, it considers the average of the second moments of the gradients, which is expected to be computational efficient with lower memory requirements.

As shown in Eq. (12), the gradients of the loss function can be calculated as $g_t = \nabla_\theta L(\theta) = \mathbb{E}_{s_t, a_t, r_t, s_{t+1}}\left[\left(y - Q(S_t, A_t;\ \theta)\right)\nabla_\theta Q(S_t, A_t;\ \theta)\right]$. We use this gradient information to update the biased first moment estimate with $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ and biased second raw moment estimate with $v_t = \beta_2 v_{t-1} + (1-\beta_2) {g_t}^2$. β_1 and β_2 represent the exponential decay rates for the first moment estimates, and the second moment estimates, respectively. Essentially, the algorithm calculates an exponential moving average of the gradient and the squared gradient, and the parameters β_1 and β_2 control the decay rates of these moving averages. The initial values of the moving averages, as well as β_1 and β_2, are suggested to be close to 1.0, which will result in a bias of moment estimates towards zero.

Next, we calculated the bias-corrected first moment estimate with $\widehat{m_t} = \frac{m_t}{1-\beta_1^t}$, and the

second raw moment estimate are computed with $\widehat{v_t} = \frac{v_t}{1-\beta_2^t}$. Finally, the parameters of the neural networks are updated with $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\widehat{v_t}+\varepsilon}}\widehat{m_t}$. In this way, the temporal-difference target will be update more efficiently than traditional tabular methods.

4.4 Solution algorithm

Based on the above modules, the ADRL-MTF model is proposed to work as follows:

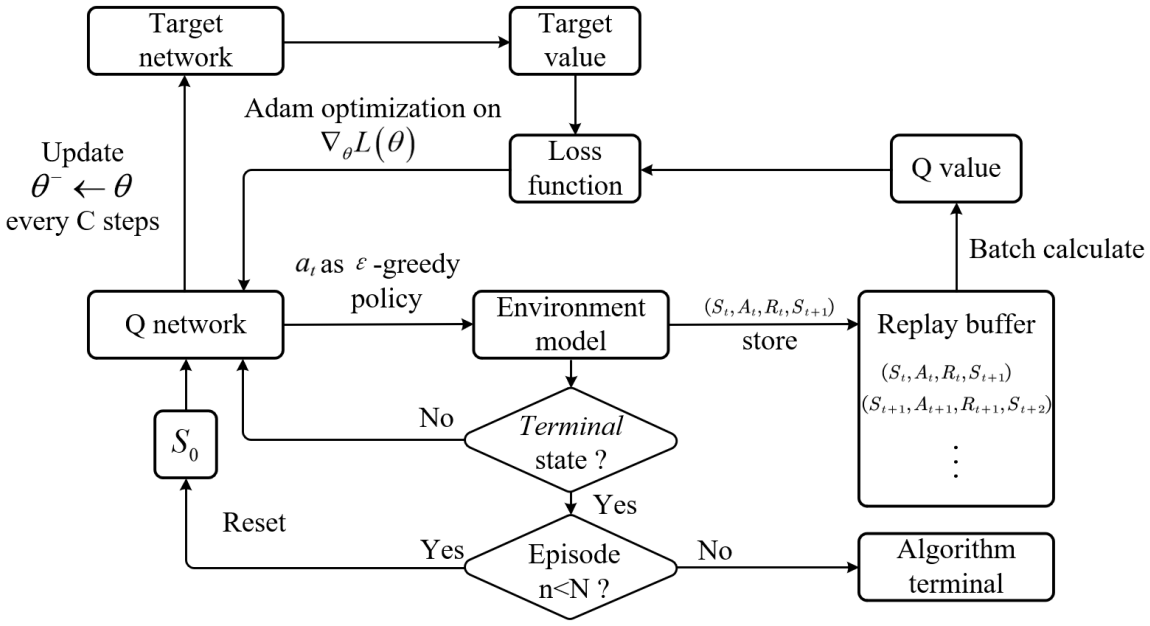


Figure 5. The process of ADRL-MTF model

The model works in the following steps.

1. At each training episode, a Q-network with initialized parameters selects an action based on ε-greedy policy, which balances exploration and exploitation at the initial state S_0 (i.e. mixed traffic flow at the first iteration). The target network is also initialized with the same parameters.
2. The environment model is activated to update the traffic flow status, and samples a tuple (S_t, A_t, R_t, S_{t+1}) to be stored in replay buffer.
3. When a sufficient number of tuple data is stored in replay buffer, a mini-batch will be executed to calculate the Q value. Then, a target value from the target network is combined with Q value and loss function, to update parameters θ of Q network through Adam optimization algorithm.
4. Target network parameter θ^- is replaced with θ from Q network every C steps.
5. On the other hand, following the step 2, the environment model also to determine whether the state is a terminal state. If yes, the environment model will be reset to initial

state and starts a new episode. If not, state transition will be continued until the end of the episode.

6. When a sufficient number of training episode is reached, the algorithm ends and the trained Q network is stored.

Table 3 below shows the pseudocode of the training algorithm.

Table 3. pseudocode of the proposed ADRL-MTF model

Decompose the platoon to be several basic interactive units
Initialize replay buffer with size D
Initialize Q network with random parameters θ, and target network with $\theta^- = \theta$
For episode=1 to N **do**
  Reset environment model: $S_0 = (c, \frac{t_{remaining}}{t_c}, 1 - \frac{y_{CAV}^t}{l_{seg}+l_a}, \frac{v_{CAV}^t}{v_s})$
  For t=1 to T **do**
    With probability ε select a random action a_t
    Otherwise select $a_t = \underset{a}{argmax}\, Q(S_{t+1}, a; \theta)$
    Environment model executes a_t, triggers car-following model, and samples reward r_t
    Generate the next state s_{t+1}
    Store (S_t, A_t, R_t, S_{t+1}) in the replay buffer D
    Sample random mini batch of (S_t, A_t, R_t, S_{t+1}) from D
    Set $y = R_{t+1} + \gamma \underset{a}{max}\, Q(S_{t+1}, a; \theta^-)$, set terminal=false
    or $y = R_{t+1}$, set terminal=true
    Calculate gradient descent and apply Adam algorithm to update θ
    every C steps, update $\theta^- \leftarrow \theta$
  End for
End for

5 CASE STUDY

In this section, the proposed ADRL-MTF model is implemented and tested on a typical arterial roadway segment with signal control. The main input data required by the training model is the length of the roadway segment, speed limit, and the signal control information. In other words, as long as the basic information of a roadway segment is known, the proposed ADRL-MTF model can perform reinforcement learning by itself and identify the best actions for CAV along the way. We compare the model's performance with a prior benchmark model from the research team based on Monte Carlo Tree Search. We also test the algorithm's stability with regards to different headway distribution and market penetration rates, to make sure the proposed model can work

with different traffic flow patterns. Sensitivity analysis is also performed to test the algorithm's effectiveness with different roadway geometries.

5.1 Experiment setup

The target scenario consists of a 400-meter roadway with a signal-controlled intersection. After decomposition, CAV becomes the leading vehicle with a platoon of following HDVs. The platoon has six vehicles that are evenly distributed near the roadway entrance. This scenario is shown in Figure 6, and the specific parameters are listed in Table 4.

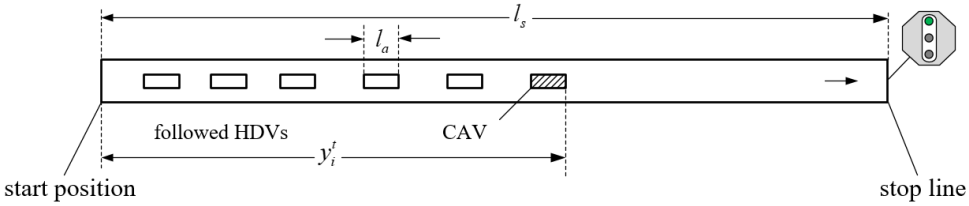


Figure 6. Experiment setup

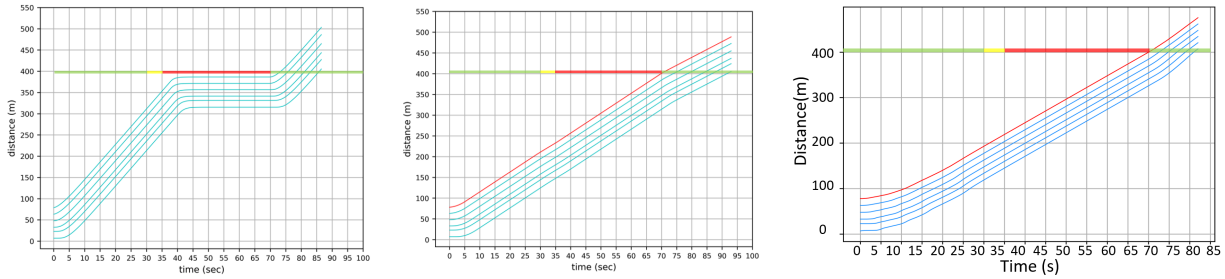
For the Q-network and target network, we use a BPNN structure with 7 layers, which includes 1 input layer, 1 output layer and 5 hidden layers. The number of nodes in the input layer is 4, which is the consistent with the state definition as described in Eq. (1). The number of nodes in the output layer equals to the number of CAV actions, which we discretize to a total of 15 different acceleration values. The CAV's speed is set to vary within $(0, v_s)$ in which v_s is the speed limit. Each hidden layer has 256 nodes, and both hidden layers and output layer use Rectified Linear Unit (ReLU) activation function which is commonly used to accelerate the convergence of stochastic gradient descent. Other hyper parameters are shown in Table 4.

Table 4. Environment variables and hyper parameters

Variables and hyper-parameters	Descriptions and values
l_{seg}	Roadway length, 400 meters
l_a	Average vehicle length, 5 meters
v_s	30 kph
Signal cycle	$t_g = 30s, t_y = 5s, t_r = 35s, t_c = 70s$
α	Learning rate, decreases from 0.01 to 0.0001 as episode increases
γ	Decay rate, 0.9999
ε	The value of ε-greedy policy, decreases from 0.1 to 0.01 as episode increases
C	Target network parameters update iterations. C=150
Batch size	50
Buffer size	10e6

5.2 Algorithm performances

We use the total fuel consumption minimization as the objective function, and find the fuel consumption drops from 709.92ml in the benchmark scenario (without any CAV intelligence) to 654.54ml with the proposed algorithm. In other words, the fuel consumption is reduced by 7.8%. This benefit is more significant than the previous MCTS-based approach, which achieves a fuel consumption saving of 6.5% (Cheng et al. 2020). The capabilities of fuel consumption are also evidenced by the time-space diagram below.



(a)Benchmark acenario (b) MCTS-based method (c) ADRL-MTF model

Figure 7. Time-Space Diagram Comparison

In Figure 7, subfigure (a) represents the benchmark scenario without any CAV intelligence, in which we can see the vehicles firstly drive at a constant high speed to the intersection, then brake and stop at the intersection due to red light, and finally accelerate and pass intersection when the light turns green. These drastic braking and acceleration cause significant fuel consumptions and greenhouse gas emissions. On the contrary, a much smoother trajectory is found in both subfigures (b) and (c), for the previously developed MCTS-based method, and this proposed ADRL-MTF model, respectively. The avoidance of sharp deceleration and acceleration ensure that fuel consumption of mixed traffic is kept as low as possible.

5.3 Runtime efficiency analysis

The training is done in a computer with Intel(R) Core (TM) i7-4510U 2.60GHz CPU and 8GB memory. We set 50 thousand episodes in the offline training, and the time consumption of this test scenario is about 5-16 hours as shown in Figure 8. The training time is mostly spent on the loss function optimization. On the other hand, with the increases of training time, the amount of data in the replay buffer is also increasing, which leads to a longer time of selecting tuple data for NN parameters updates.

Name	Smoothed	Value	Step	Time	Relative
20201022-193705	581.1	581.2	50k	Fri Oct 23, 04:19:45	8h 41m 3s
20201023-122951	646.8	648.1	50k	Fri Oct 23, 23:20:15	10h 49m 30s
20201024-002950	1215	1216	50k	Sat Oct 24, 14:30:37	13h 59m 51s
20201025-165339	396.5	397	50k	Mon Oct 26, 02:52:55	9h 58m 16s
20201028-012250	137.9	137.7	50k	Wed Oct 28, 07:05:58	5h 42m 8s
20201028-120206	141.4	141.2	50k	Wed Oct 28, 19:27:01	7h 24m 0s
20201028-235308	714.2	714.4	50k	Thu Oct 29, 12:49:17	12h 55m 4s
20201124-232352	592	591.2	50k	Wed Nov 25, 11:30:26	12h 5m 40s
20201126-234942	1174	1175	50k	Fri Nov 27, 15:56:27	16h 5m 34s
20201127-172120	882.5	882.8	50k	Sat Nov 28, 08:02:47	14h 40m 28s
20201130-170250	496.1	496.4	50k	Tue Dec 1, 05:03:42	11h 59m 56s
20201201-122633	500	500	50k	Wed Dec 2, 00:54:13	12h 26m 48s

Figure 8. List of training time

However, once trained, the proposed model can be used to guide CAV trajectory planning in real time with very little runtime requirement. In our experiment, when we load the trained model and calculate the total run time of the program for the test scenario, the required run time is 1.06 second for the entire planning horizon. Note this is the total runtime for the entire roadway segment of 400m, which has a travel time of 93 seconds. During the time discretization, we set the time step to be 0.5 second, which means there are a total of 186 time intervals. As a result, we can find that the average time needed for CAV to determine one step of movement is 1.06/186=0.0057second, suggesting this algorithm is promising for real time applications. For comparison purpose, the previously proposed MCTS model spends about 0.89 second to decide the optimal movement policy in one time interval. In other words, this proposed algorithm is able to speed up the CAV trajectory planning by 100 times.

5.4 Stability testing

The purpose of this section is to test the stability of algorithm outcomes, when dealing with different headway distribution and CAV market penetration rates. The purpose is to make sure the proposed model can work with different traffic flow patterns. Note that stability testing does not include roadway geometry (e.g., link length or speed limit) or signal timing plan, otherwise the model will have to be trained again which will be discussed in the sensitivity analysis in the next section.

5.4.1 Headway distribution change

We test two types of headway distributions for this mixed traffic flow, namely, uniform headway distribution and normal headway distribution. Both distributions are set with a higher demand scenario and a lower demand scenario, respectively. For example, in Table 5 below, $U(12,16)$ means the headway is subject to a uniform distribution with a lower bound of 12 meters and upper bound of 16 meters, and $N(12,2)$ means the headway is subject to a normal distribution with a

mean of 12 meters and standard deviation of 2 meters. For each scenario, we calculate the fuel consumption of the benchmark model without any CAV intelligence, and of the proposed ADRL-MTF model, respectively. It can be found that for all headway distribution scenarios that are analyzed below, the fuel consumption savings range between [8%, 9%] and does not fluctuate much. In other words, the performance of the proposed model remains stable regardless of the traffic flow headway distribution patterns.

Table 5. Performance with initial distance headway distribution change

$d_{hi}^{t=0}$ distribution	$d_{hi}^{t=0} \sim U(12,16)$	$d_{hi}^{t=0} \sim U(10,18)$	$d_{hi}^{t=0} \sim N(12,2)$	$d_{hi}^{t=0} \sim N(14,4)$
Benchmark FC (ml)	709.92	709.05	708.45	713.75
ADRL-MTF FC (ml)	650.29	652.06	645.72	653.17
FC Savings	8.40%	8.04%	8.85%	8.49%

5.4.2 Market penetration rates

We also test the fuel consumption savings with different ratios of CAVs in the mixed traffic flow, and the results are shown in Figure 9 below. It can be seen that when all vehicles are CAVs (i.e., MPR=100%), the maximum fuel consumption savings is observed and the number is 11.94%. When the MPR decreases, the fuel consumption saving reduces but always remains a positive number. The reasons of this decreasing trend might be that as CAV MPR reduces, the number of HDVs following CAV will increase, and the shockwave generated by the leader CAV will take a longer time to propagate to the tail of the HDV platoon, and as a result the effect on fuel consumption savings is weakened. When the CAV MPR is 11%, i.e., 1 CAV and 8 following HDVs (or 1 CAV in 9 total vehicles in the mixed traffic flow), the fuel consumptions savings are still relatively high at 4.7%. However, if the CAV MPR continues to drop, the fuel consumption savings will become less significant.

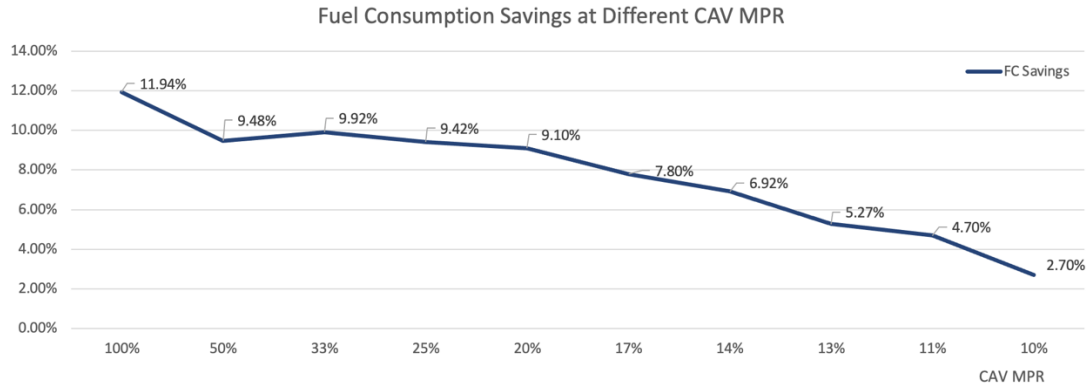


Figure 9. Performance with different market penetration rate

5.5 Sensitivity analysis

In this section we perform sensitivity analysis to test how will the algorithm performance vary, if

the roadway conditions change. In particular, we focus on the link length and the speed limit. It should be noted that when roadway geometry change, the environment model changes and thus we need to re-train the model.

5.5.1 Link length

We use the same network setup as Table 4, but increase the length of the roadway segment from 400 meters to 500 meters and then to 1,000 meters. The vehicle headway distribution pattern, speed limitation and the signal timing plan are all kept the same, and the results are shown in Table 6. It can be found that as the link length increases, the fuel consumption saving reduces slowly from 7.8% to 5.06%, and then to 4.83%. This might be due to the increases of the denominator, as when the link length increases the benchmark travel time is also higher, and so is benchmark fuel consumption. As a result, the fuel consumption saving percentage becomes lower.

Table 6. Results in different link length with the same speed limit

Roadway conditions		Benchmark FC (ml)	ADRL-MTF FC (ml)	FC Savings
v_s=30kph	l_s=400m	709.92	654.54	7.8%
	l_s=500m	769.57	730.61	5.06%
	l_s=1000m	1436.06	1366.68	4.83%

5.5.2 Speed limit

We test two scenarios, with link length being 500 meters and 1,000 meters, respectively. For the former case, we slightly increase the speed limit from 30kph to 36kmp, and the fuel consumption saving increases from 5.06% to 6.56%. For the latter case, we increase the speed limit significantly from 30kph to 60kph, and the fuel consumption saving increases from 4.83% to 13.72%. In other words, the fuel consumption savings are consistently higher in both cases when the speed limit is higher. It is suspected that two reasons might be playing a role. First, when the speed limit becomes higher, the denominators might be lower (as benchmark travel time and fuel consumption is now lower), and as such the percentage of fuel consumption saving becomes higher. Second, which we suspect might be more important, is that a higher speed limit suggest more rooms to CAV to adjust its speed (i.e., more allowable actions to choose from, in the MDP problem), leading to a much improved results.

Table 7. Results in different speed limit with the same link length

Roadway conditions		Benchmark FC (ml)	ADRL-MTF FC (ml)	FC Savings
l_s=500m	v_s=30kph	769.57	730.61	5.06%
	v_s=36kph	790.02	738.16	6.56%
l_s=1,000m	v_s=30kph	1436.06	1366.68	4.83%
	v_s=60kph	1246.81	1075.79	13.72%

6. CONCLUSION

An Adam optimization-based Deep Reinforcement Learning model for Mixed Traffic Flow control, or ADRL-MTF model, is developed in this manuscript to guide CAV's longitudinal trajectory control, to minimize fuel consumption of a vehicle platoon. When compared with the literature, the proposed model does not require the common assumptions to simplify the problem solving, but in the meantime is able to run efficiently and with improved modeling realism. ε-greedy is used to balance between exploration and exploitation, and two neural networks are developed to replace the traditional Q function and generate the targets in the Q-learning update.

The proposed model is implemented and tested on a typical urban arterial roadway segment with a signal-controlled intersection. The result shows that when compared with benchmark scenario which does not have any CAV intelligence, the proposed model is able to reduce fuel consumption by 7.8%. Compared with a MCTS-based approach that was previously developed by the research team, the fuel consumption saving is more significant. Further, it is also found that the average time needed for CAV to determine one-step movement is 0.0057second, which is about 100 times faster than the previous MCTS-based approach and suggests promising potentials to support real time applications. Stability testing is also performed, and find that the algorithm works well with different headway distributions, and works the best when the CAV market penetration rate is over 11%, i.e., at least 1 CAV in every 9 vehicles in the mixed traffic. Sensitivity analysis is also performed, and find that fuel consumption saving percentage increases when the link length is reduced, or when the speed limit of the roadway increases. These results suggest that the developed model outperforms a previously developed benchmark model, and as long as the roadway geometry (i.e., the roadway length and speed limit) and signal timing plan are given, the developed model can perform reinforcement learning by itself and guide CAV's longitudinal movement in real time.

7. REFERENCE

Ahn, K., Rakha, H. A. and Park, S. (2013). "Ecodrive Application." Transportation Research Record: Journal of the Transportation Research Board 2341(1): 1-11.

Ahn, K., Rakha, H. A. and Park, S. (2013). Ecodrive Application: Algorithmic Development and Preliminary Testing.

Cao, Z., Jiang, S., Zhang, J. and Guo, H. (2017). "A Unified Framework for Vehicle Rerouting and Traffic Light Control to Reduce Traffic Congestion." IEEE Transactions on Intelligent Transportation Systems 18(7): 1958-1973.

Cheng, Y., Hu, X., Tang, Q., Qi, H. and Yang, H. (2020). "Monte Carlo Tree Search-Based Mixed Traffic Flow Control Algorithm for Arterial Intersections." Transportation Research Record 0(0): 0361198120919746.

Eskandarian, A., Wu, C. and Sun, C. (2019). "Research Advances and Challenges of Autonomous and Connected Ground Vehicles." IEEE Transactions on Intelligent Transportation Systems: 1-29.

Ghiasi, A., Li, X. and Ma, J. (2019). "A mixed traffic speed harmonization model with connected autonomous vehicles." Transportation Research Part C: Emerging Technologies 104: 210-233.

Guanetti, J., Kim, Y. and Borrelli, F. (2018). "Control of connected and automated vehicles: State of the art and future challenges." Annual Reviews in Control 45: 18-40.

He, X., Liu, H. X. and Liu, X. (2015). "Optimal vehicle speed trajectory on a signalized arterial with consideration of queue." Transportation Research Part C: Emerging Technologies 61: 106-120.

Jiang, H., Hu, J., An, S., Wang, M. and Park, B. B. (2017). "Eco approaching at an isolated signalized intersection under partially connected and automated vehicles environment." Transportation Research Part C: Emerging Technologies 79: 290-307.

Kingma, D. P. and Ba, J. (2015). "Adam: A Method for Stochastic Optimization." CoRR abs/1412.6980.

Lee, J. and Park, B. (2012). "Development and Evaluation of a Cooperative Vehicle Intersection Control Algorithm Under the Connected Vehicles Environment." IEEE Transactions on Intelligent Transportation Systems 13(1): 81-90.

Liang, X., Guler, S. I. and Gayah, V. V. (2019). "Joint Optimization of Signal Phasing and Timing and Vehicle Speed Guidance in a Connected and Autonomous Vehicle Environment." Transportation Research Record 2673(4): 70-83.

Liang, X., Guler, S. I. and Gayah, V. V. (2020). "An equitable traffic signal control scheme at isolated signalized intersections using Connected Vehicle technology." Transportation Research Part C: Emerging Technologies 110: 81-97.

Liang, X., Guler, S. I. and Gayah, V. V. (2020). "Traffic Signal Control Optimization in a Connected Vehicle Environment Considering Pedestrians." Transportation Research Record 2674(10): 499-511.

Ma, J., Li, X., Zhou, F., Hu, J. and Park, B. B. (2017). "Parsimonious shooting heuristic for trajectory design of connected automated traffic part II: Computational issues and optimization." Transportation Research Part B: Methodological 95: 421-441.

Mirheli, A., Hajibabai, L. and Hajbabaie, A. (2018). "Development of a signal-head-free intersection control logic in a fully connected and autonomous vehicle environment." Transportation Research Part C: Emerging Technologies 92: 412-425.

Mirheli, A., Tajalli, M., Hajibabai, L. and Hajbabaie, A. (2019). "A consensus-based distributed trajectory control in a signal-free intersection." Transportation Research Part C: Emerging Technologies 100: 161-176.

Walraven, E., Spaan, M. T. J. and Bakker, B. (2016). "Traffic flow optimization: A reinforcement learning approach." Engineering Applications of Artificial Intelligence 52: 203-212.

Wang, M., Daamen, W., Hoogendoorn, S. P. and van Arem, B. (2014). "Rolling horizon control framework for driver assistance systems. Part I: Mathematical formulation and non-cooperative systems." Transportation Research Part C: Emerging Technologies 40: 271-289.

Wu, X., He, X., Yu, G., Harmandayan, A. and Wang, Y. (2015). "Energy-Optimal Speed Control for Electric Vehicles on Signalized Arterials." IEEE Transactions on Intelligent Transportation Systems 16(5): 2786-2796.

Yang, K., Guler, S. I. and Menendez, M. (2016). "Isolated intersection control for various levels of vehicle technology: Conventional, connected, and automated vehicles." Transportation Research Part C: Emerging Technologies 72: 109-129.

Yao, H., Cui, J., Li, X., Wang, Y. and An, S. (2018). "A trajectory smoothing method at signalized intersection based on individualized variable speed limits with location optimization." Transportation Research Part D: Transport and Environment 62: 456-473.

Zhou, F., Li, X. and Ma, J. (2017). "Parsimonious shooting heuristic for trajectory design of connected automated traffic part I: Theoretical analysis with generalized time geography." Transportation Research Part B: Methodological 95: 394-420.

Zhou, M., Yu, Y. and Qu, X. (2020). "Development of an Efficient Driving Strategy for Connected and Automated Vehicles at Signalized Intersections: A Reinforcement Learning Approach." IEEE Transactions on Intelligent Transportation Systems 21(1): 433-443.